Project 1: What Does Your Vision Model See?

Chongxu He
Johns Hopkins University
che44@jh.edu

Hanyu Qiu
Johns Hopkins University
hqiu21@jh.edu

Abstract

We compare ten classifiers on object crops from 19 Pascal VOC 2012 classes, from a linear model on pixels up to a pretrained ViT-T/16, and then test three of them on our own photos of five classes. In Part A, hand-crafted features already beat raw pixels (43.7% vs. 16.1% mean per-class test accuracy), a small CNN trained from scratch beats a ViT of about the same size (55.0% vs. 34.0%), and the pretrained ViT-T/16 gets 92.2%, losing only 1.3 points when its input is a 64 pixel crop upsampled to 224. In Part B, all models do worse on our own photos than on VOC, but A6-H still gets 86.0% (vs. 93.5% on VOC for the same classes), while A2 and A4 fall to 27.7% and 41.7%. In our controlled series, distance hurt A6-H the most (0.9 to 0.5 when the object is far), more than rotation or occlusion. One short fine-tuning run on far photos raised A6-H on new far objects from 0.4 to 0.8 and cost 0.5 points on the VOC test set. For the 665 extension, a frozen ViT with only a linear head gets 88.6% on VOC (3.6 points below fine-tuning) but 90.0% on our photos, 4 points above the fine-tuned model.

1. Setup

We use object crops from Pascal VOC 2012 [1], made by the starter code. It keeps every box that is at least 32 pixels on each side and not marked difficult, cuts a square of side $\max(w, h)$ around the center of the box, fills any part outside the photo by repeating the border, and resizes it. Person is left out, so we have 19 classes and 16,719 crops. The test set is VOC `val`. The validation set is all crops from a random 10% of the VOC `train` images (seed 0), so the split is by image. We checked that no image ID appears in two splits (all three pairs share 0 images). Every model is trained and tested on all 19 classes, and it always picks one of the 19.

Our five classes are bicycle, bottle, chair, car, and cat. We can find several of each around campus or at home, and they have quite different amounts of training data, from about 300 crops (bicycle, bottle) to about 1,000 (chair). Table 1 gives the counts. We report mean per-class accuracy, $C^{-1} \sum_c \text{correct}_c/\text{total}_c$, with $C = 19$ on VOC and $C = 5$ on our classes.

class	train	validation	test
bicycle	312	23	344
bottle	299	35	386
chair	1005	102	1093
car	681	76	754
cat	546	58	608
all 19 classes	7541	797	8381

Table 1. Crops per split for our five classes and in total.

Preprocessing. Every model gets the same square crops, with no extra cropping. A1 and A3 use the 64 pixel crops resized to 32×32 (OpenCV, area interpolation). A2, A4, and A5 use 64×64, A6-H uses the 224×224 crops, and A6-L uses the 64 pixel crops upsampled to 224 (bilinear). All normalization statistics come from the training crops only. For A1 and A3 we standardize each color channel with the mean and std of the 32 pixel training crops, and for A4 and A5 with those of the 64 pixel training crops. Using one value per channel keeps the spatial structure of the image. For A2 we standardize each feature with its training mean and std, separately for each feature block, before putting the blocks together. The A2 features are a 48-d HSV histogram (16 bins per channel, each normalized to sum to 1), a 1764-d grayscale HOG (`cv2.HOGDescriptor` with 8×8 cells, 16×16 blocks, stride 8, and 9 bins), and a 16-d Gabor feature: the mean and variance of the absolute response of 8 zero-mean `cv2.getGaborKernel` filters (4 orientations, wavelengths 4 and 8 pixels, $\sigma = 0.56\lambda$, kernel sizes 13 and 27, aspect ratio 0.5, phase 0). A6 uses the checkpoint's own normalization (mean and std 0.5 per channel on the 0–1 scale). The only augmentation is a random horizontal flip with probability 0.5, used for A4, A5, A6, and the B5 repair.

Training. Every model minimizes 19-way softmax cross-entropy with a constant learning rate, no

dropout, a new shuffle of the training data every epoch, and seed 0 (`torch.manual_seed(0)` and `np.random.default_rng(0)`) before each run. We keep the epoch with the best validation mean per-class accuracy over 19 classes and test that model once. We used the reference settings from the handout without changes. A1 and A2 use Adam, learning rate 10^{-3}, batch 256, 60 epochs, zero initialization, and an L2 penalty on the weights only ($\lambda \in \{10^{-2}, 1, 10\}$ for A1 and $\{10^{-5}, 10^{-3}\}$ for each A2 feature set, picked on validation). A3 and A4 use AdamW with learning rate 10^{-3}, weight decay 10^{-4}, batch 64, and 30 epochs. A5 uses AdamW with 3×10^{-4} and weight decay 10^{-2}. A6 uses AdamW with 10^{-5} for the encoder and 10^{-3} for a new 19-way head (created right after `torch.manual_seed(1)`), weight decay 10^{-2}, batch 32, and 5 epochs. For AdamW, the weight decay applies to all parameters. In A5, the class token and the position embeddings start from a truncated normal with std 0.02. Before the full runs, we checked that A4 can overfit 64 training crops (150 updates, no flips, AdamW 10^{-3}): it got all 64 right (100%). All models were trained on one NVIDIA Tesla T4 GPU in Google Colab (PyTorch 2.11, timm 1.0.30, OpenCV 4.14).

2. Part A: a ladder of classifiers

Predictions. These are our milestone predictions, copied without changes:

> We think the order will be A6-H > A6-L > A4 > A5.
>
> A6-H (80–90%): it was pretrained on ImageNet-21k and ImageNet-1k, which already include most VOC classes, and it gets the crops at 224 pixels like in pretraining. So fine-tuning only needs to adjust it a bit.
>
> A6-L (65–80%): it is the same pretrained model, but the 64 pixel crops are blurry after upsampling to 224, so a lot of fine detail is lost. We expect thin or small objects like bicycles, bottles, and chairs to suffer the most.
>
> A4 (40–55%): we only have 7,541 training crops and no pretraining. Convolutions should still learn useful edge and texture filters, so we guess it will end up around our A2 concatenation result (48.52% on validation), but probably still mix up similar classes like cat and dog or chair and sofa.
>
> A5 (25–40%): a ViT of a similar size has no built-in idea of locality, so it has to learn local filters and the 2D layout from scratch with the same small dataset. We expect it to overfit faster than A4.
>
> CNN features: we expect the first layer to look like oriented edges, color blobs, and Gabor-like patterns, because those are useful for every class. In the last stage we expect channels that respond to textures and parts like wheels, fur, or chair legs, and probably also to background that often comes with a class (road, grass, indoor walls), since at 64×64 background is an easy shortcut.

We got the order right: A6-H (92.2%) > A6-L (90.9%) > A4 (55.0%) > A5 (34.0%). A4 and A5 are inside our ranges, and A4 is right at the top of its range. We guessed A4 would be about as good as the A2 concatenation, but it was 11.3 points better. Both pretrained models did better than we guessed. A6-L was our biggest mistake: we thought upsampling would cost 10 to 25 points, but it only cost 1.3. Our reason was only partly right. Bicycle (-4.1) and bottle (-3.4) did get worse at 64 pixels, but cat got worse the most (-5.6), car almost did not change (-0.5), and chair even got a little better ($+1.0$). So the pretrained model can still find most of what it needs in a blurry 64×64 crop, and the lost detail seems to matter more for fur than for thin parts. For the CNN features, the first layer did not become clean edge filters like we expected. After training, A4's kernels still look mostly like their random starting values, and what changed is smooth color and brightness patterns (V4a). The later stages did look like we predicted: stage 3 has channels for animal faces and for rectangular frames like screens, and one stage 1 channel reacts to green grass, which shows up with horses, cows, and dogs (V4c).

Results. Table 2 shows the test results. The linear model on pixels (A1, $\lambda = 1$) gets 16.1%, about three times chance, but it is almost always wrong on bicycle and bottle and mostly guesses the big classes, chair and car. For the hand-crafted features, HOG alone (39.4%) is much better than color (HSV, 14.4%) or Gabor (20.4%), and putting all three together is best (43.7%). So the shape of the edges matters much more than color, and color and texture only help a little, mostly for car and cat. The MLP (A3, 23.7%) uses the same pixels as A1 and is 7.6 points better, but it is still worse than HOG. The CNN trained from scratch (A4, 55.0%) is the first model that beats the hand-crafted features. Its biggest gains are on bottle (67.6%, compared to 36.3% for A2) and car (85.7%). The ViT trained from scratch (A5, 34.0%) has about the same size as A4 but is 21.0 points worse, and it is even worse than A2. A5 is not worse on every class though: on cat it beats A4 (43.4% vs. 36.5%). Cat is the one class of ours that every model trained from scratch is bad at. Pretraining makes a huge difference: both A6 models are above 90%, and cat (96.9% for A6-H) and bottle (96.6%) are now their easiest classes. We think the scratch models mix up cat and dog, since dog is a similar class with 672 training crops, but we did not check a confusion matrix, so this is only a guess.

Fair comparisons and diagnosis. None of our comparisons change only one thing. A1 and A3 have the same input, but A3 is deeper and also uses a different optimizer

	model	mean per-class (19)	accuracy on our five classes					input	params / dims	training data	time, hardware
			bicycle	bottle	chair	car	cat				
A1	linear, pixels	16.1	0.0	0.8	65.0	44.8	9.2	32^2	58,387	VOC	6 s, T4
A2	HSV histogram	14.4	3.8	1.8	53.3	51.3	13.2	64^2	48-d / 931	VOC	3 s, T4
	HOG	39.4	41.3	33.9	47.9	56.2	26.5	64^2	1764-d / 33,535	VOC	4 s, T4
	Gabor	20.4	15.1	0.0	66.0	59.2	20.4	64^2	16-d / 323	VOC	3 s, T4
	concatenation	43.7	48.0	36.3	53.3	60.9	30.1	64^2	1828-d / 34,751	VOC	5 s, T4
A3	MLP	23.7	13.4	15.8	34.4	32.5	15.0	32^2	791,571	VOC	7 s, T4
A4	CNN, scratch	55.0	61.0	67.6	56.0	85.7	36.5	64^2	701,619	VOC	275 s, T4
A5	ViT, scratch	34.0	27.3	28.8	39.3	50.0	43.4	64^2	828,947	VOC	68 s, T4
A6-H	ViT-T/16, pretrained	92.2	92.4	96.6	87.4	94.3	96.9	224^2	5,528,083	IN-21k/1k + VOC	167 s, T4
A6-L	ViT-T/16, pretrained	90.9	88.4	93.3	88.4	93.8	91.3	$64^2 \to 224^2$	5,528,083	IN-21k/1k + VOC	166 s, T4

Table 2. Test results on the VOC 2012 `val` crops (8,381 crops), mean per-class accuracy in %. VOC training data is 7,541 crops for every model; A6 was also pretrained on ImageNet-21k and fine-tuned on ImageNet-1k. Times are total training time on one Tesla T4, validation included; A2's time excludes feature extraction. A2's parameter count is its linear classifier on the given feature dimension. Chance is 5.3%.

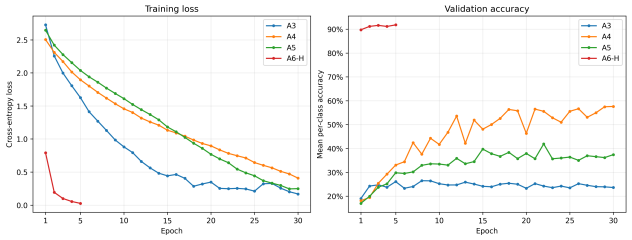


Figure 1. Training loss (left) and validation mean per-class accuracy over 19 classes (right) per epoch for A3, A4, A5, and A6-H.

(AdamW instead of Adam with an L2 penalty), batch size, and number of epochs. A1 and A2 use the same classifier, but A2 looks at 64 pixel crops through fixed features. A4 and A5 have the same input, flips, and epochs, and similar sizes (0.70M and 0.83M parameters), but besides the architecture they also use different learning rates and weight decay. A5 and A6-H differ in pretraining, but A6-H is also much bigger (5.5M parameters) and uses bigger patches, bigger images, and fewer epochs, so their gap is not only from pretraining. A6-H vs. A6-L is the cleanest comparison: same model, same pretraining, and only the image detail changes.

Before the full runs, A4 got 100% (64/64) on 64 training crops after 150 updates. So our data, model, and training loop work, and the bad results below are not because the model cannot be trained. The curves (Fig. 1) show different behavior for each model. A3's validation accuracy is best at epoch 8 (about 26.5%) and then slowly goes down, while its training loss keeps going down to about 0.17. This means A3 is memorizing the training crops, which makes sense for a fully connected layer with 0.8M weights and only 7,541 images. A4 is still improving at the end: its best epoch is the last one (57.6% at epoch 30) and its training loss is still 0.41, so it is not memorizing yet, and more epochs would probably help. A4's validation accuracy jumps around a lot (for example 53.6% at epoch 12 and 42.2% at epoch 13). Part of the reason is that the validation set is small: some classes only have 15 to 30 validation crops, so a few crops can change the mean by several points. A5's training loss goes down slower than A4's at first but ends lower (0.25), while its validation accuracy stays around 35–40% after epoch 15 (best at epoch 22, 41.9%). So A5 learns the training data but does not generalize well, which is overfitting and not a training problem. A6-H is already near 90% after one epoch and only gains 2 more points, while its training loss drops from 0.79 to 0.03. So most of the work is done by the pretrained features. Training time does not follow accuracy: A4 took 275 s and A5 only 68 s, because A4's first stage works on the full 64×64 image while A5 only has 64 tokens. A6-H took 167 s for 5 epochs.

Looking inside (V1–V5). Figure 2 shows A1's weights for our five classes at the three L2 settings, each from its best validation epoch and each scaled to its own range, $(w - \min w)/(\max w - \min w)$. Above them is each class's mean training crop minus the mean of all training crops. With $\lambda = 10^{-2}$ the weights look like noise, except for car, where a horizontal band already shows up. With $\lambda = 1$ they become smooth and look like the class means: car has a horizontal body with a dark band where the wheels are, bottle has a bright shape in the middle, and chair has a reddish color. With $\lambda = 10$ they get even blurrier. The easiest one to read is not the best one, though: validation accuracy drops from 23.77% to 18.65% to 14.16% as λ goes up. A linear model can only store one template per class, so a stronger penalty pushes it toward the average image and it fits the data less.

V2 (Fig. 3) shows A2's Gabor filters and the HOG histograms of three validation crops without people (one bicycle, one bottle, one chair, picked with seed 42 before we looked). We computed the HOG histograms ourselves: Sobel gradients, 9 orientation bins of 20°, and a magnitude-weighted `np.bincount` in every 8×8 cell. Each bin is

drawn as a line along its edge direction, and the line length is relative to the biggest bin in the crop. The Gabor filters are shown with gray at zero, $0.5 + 0.5\,w/\max|w|$. HOG keeps the main edge direction in each cell but loses color and the exact position of an edge inside a cell. For example, the red bicycle frame does not stand out in HOG at all, while the strongest cells are the repeated-border stripes at the bottom of the crop and the tree trunk, which are not the bicycle. In the dark chair crop most cells are almost empty, because there are not many edges in low light. The Gabor features measure how much texture there is at two sizes (4 and 8 pixels) and four directions, but they average over the whole crop, so they lose where the texture is.

V3 (Fig. 4) shows eight first-layer units of A3, picked with seed 0 before we looked, each as a 32×32 color image scaled to its own range. They do not look like class templates the way A1's weights at $\lambda = 1$ do. Most of them look like noise with a faint colored blob (units 10 and 67), and none of them looks like one object. Together with the curves (best at epoch 8 while the training loss keeps going down), this suggests the units fit pixel patterns of single training crops instead of shapes shared by a class. A fully connected layer has its own weight for every pixel and color, so nothing forces it to reuse a pattern at different positions, and with only 7,541 crops it can just remember specific images.

V4 (Fig. 5) has three parts. (a) Eight of A4's 32 first-layer 7×7 kernels (picked with seed 0), after training and as the change from their starting values, next to the Gabor filters and the first eight first-layer kernels of an ImageNet ResNet-18. Every kernel has gray at zero, $0.5 + 0.5\,w/\max|w|$. (b) Three channels per stage (picked with seed 0: channels 19, 16, 25 in stage 1; 1, 4, 2 in stage 2; 81, 64, 115 in stage 3) on one of our photos, a FIJI water bottle on a table. Each map is scaled to its own maximum. (c) For each of these channels, the four training crops without people (out of 5,906) with the highest average activation after the ReLU. The trained kernels still look like random color noise, because training changed them only a little compared to their random start. The change itself is smooth: color and brightness gradients across the kernel (kernels 2, 13, 1) and color blobs (kernels 0, 8). This looks more like ResNet-18's color blobs (its kernels 4–6) than like the thin Gabor stripes.

On our bottle photo, the stage 1 channels react to edges and small details: the shelves on both sides, the table edge, the bottle cap, and the red flower on the label. In stage 2, channel 2 is strongest on the middle of the bottle, while channel 4 reacts mostly to the cluttered shelves and the top border. The stage 3 maps are only 16×16, and they react to the bottom of the bottle (channels 81 and 64) but just as much to the background corners and the shelves (channels 81 and 115). So even the deep channels look at the background a lot, not only at the object.

The top crops show that the features get more complex from stage to stage. Stage 1 channels are about color and simple texture: channel 19 likes bright blue (a blue monitor screen, a blue background, a blue-lit bottle), channel 16 likes high-contrast stripes and grids (window blinds, a fence, colored stripes), and channel 25 likes green grass, in crops of horses, a dog, and a cow, so it is really a background channel. Stage 2 channels mix texture and shape: dark tilted rectangles like monitors seen from the side (channel 1), soft beige surfaces like cat fur, an armchair, and a bird (channel 4), and animals standing on grass, mostly sheep (channel 2). Stage 3 channels are closer to object parts, but they mix classes: channel 64 likes animal faces and animals lying down (a kitten, a dog's face, cats and dogs), channel 115 likes a kitten's face but also a dog, a cat in a tree, and a monitor, and channel 81 likes rectangular frames, two TV screens and two chairs. So the deep channels group things by shape and texture (faces, frames), even when the classes are different, like cat and dog or tvmonitor and chair.

V5 (Fig. 6) shows, for A5 and A6-H, the first six principal components of the patch embedding filters (each filter flattened to a row of length $3p^2$, columns centered, each component shown as a $p \times p$ color patch with gray at zero, and the sign flipped so its largest value is positive). It also shows the cosine similarity between the position embedding of a central patch and all other patches, without the class token: patch (4,4) on A5's 8×8 grid and patch (7,7) on A6-H's 14×14 grid, written as (row, column) starting from 0. A5's first two components are just flat colors (green and blue), so its filters mostly differ in which color they look at, and the other components look like noise. So A5 did not learn local edge filters. A6-H's components have clear structure: fine checkerboards (PC1–2), a horizontal light-dark band (PC3), a blob in the center (PC4), and ring patterns (PC5–6), which look like Gabor and center-surround filters. Both models learned where patches are in 2D. For A5, the similarity is a smooth blob around (4,4) that fades with distance in all directions and becomes negative near the edges. For A6-H, it is a cross shape: patches in the same row and the same column are the most similar. Keep in mind that the two models differ in more than the training data (size, patch size 8 vs. 16, grid 8 vs. 14), so we cannot say which difference causes this.

Questions. *Which designed features look like the learned filters?* The pretrained ones do. ResNet-18's first layer and A6-H's patch components have edges at different angles, center-surround blobs, and ring or stripe patterns, which is what the Gabor filters capture. They also have color blobs, which is closer to the HSV histogram. A4's first layer, trained for 30 epochs on 7,541 crops, only moved toward smooth color gradients, and A5's patch filters mostly look

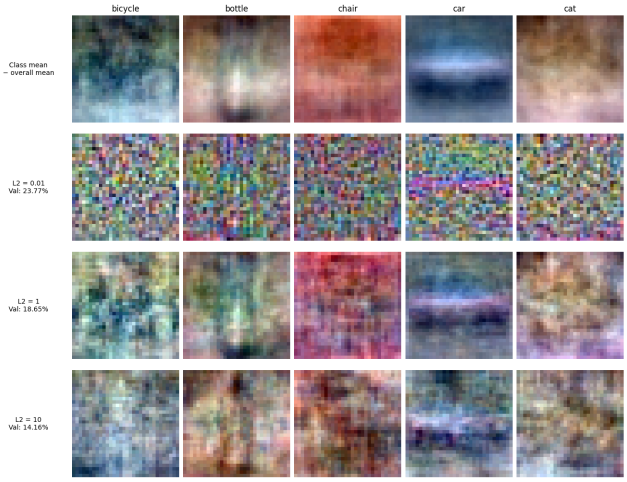


Figure 2. V1: for our five classes, each class's mean training crop minus the overall mean (top), and A1's weights at three L2 settings, each at its best validation epoch and each template on its own scale, with validation accuracy.

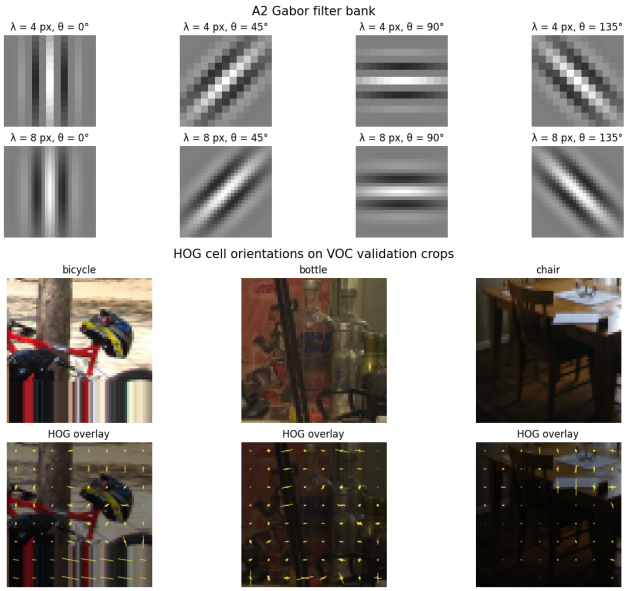


Figure 3. V2: A2's Gabor bank (gray at zero), and per-cell HOG orientation histograms on three validation crops without people (seed 42).

at color, so neither of them learned Gabor-like stripes from this small dataset.

Which representations work best? Pretrained ViT features by far (A6-H 92.2%), then CNN features learned from scratch (A4 55.0%), then hand-crafted features (A2 concatenation 43.7%). The ViT from scratch (34.0%) and the models on raw pixels (A3 23.7%, A1 16.1%) are the worst.

What shows local patterns, parts, or whole objects? A1's templates at $\lambda = 1$ are averages of whole objects (V1),

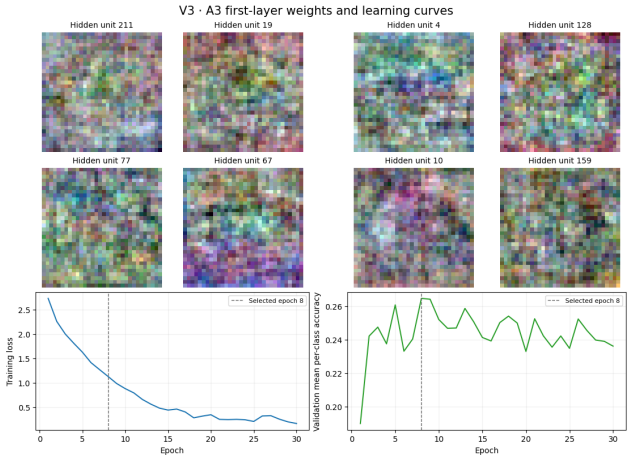


Figure 4. V3: eight first-layer units of A3 (seed 0), each on its own scale, with A3's training loss and validation accuracy (epoch 8 selected).

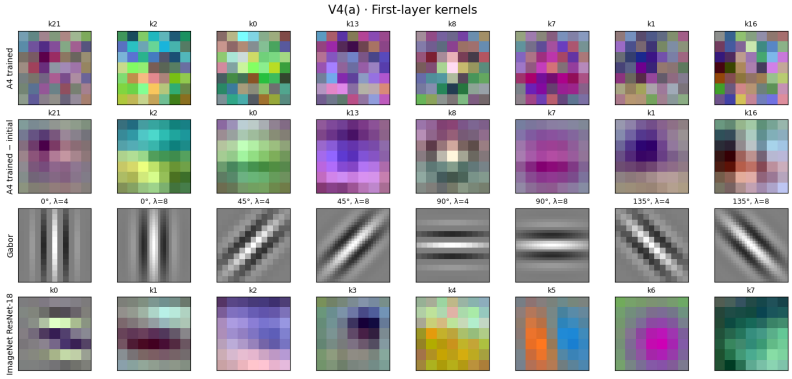
and A3's units fit pixels (V3), and both models are weak. In A4, stage 1 channels react to color and simple texture (blue, stripes, grass), stage 2 channels to texture plus shape like fur, wool, and tilted rectangles, and stage 3 channels to parts like animal faces and rectangular frames (V4c). None of the channels we looked at reacts only to whole objects. So in A4 we see local patterns that turn into parts, and it also uses background like grass along the way.

If the CNN and the ViT differ, which explanations does our evidence support? A5 can fit its training data (its training loss ends lower than A4's) but it generalizes worse than A4, and its patch filters have no local structure (V5). A4's convolutions reuse the same local filters everywhere by design. So our results fit the idea that A5 has to learn locality from only 7,541 crops and does not manage to. That A5 would catch up with A4 if it had more data or more training time is only a hypothesis, because we did not try that. A5 did learn a smooth 2D layout of its patch positions, so a missing layout is not the reason.

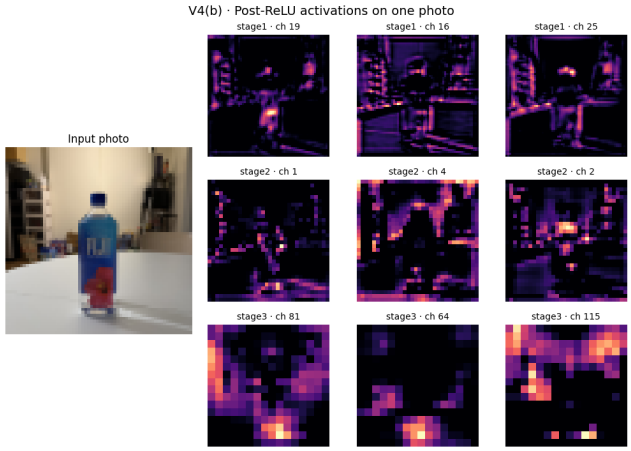
3. Part B: testing the models in our world

Baseline (B1). We took ordinary photos of our five classes with at least three objects per class and no people. The loader turns each photo upright using its EXIF tag, takes the center square, and resizes it to each model's input size, and then each model uses its own Part A preprocessing. We ended up with 52 photos instead of 50: 10 for each class except cat, which has 12, because two extra cat photos stayed in the folder when we added our third cat. Since accuracy is averaged per class, this only changes the weight inside the cat class.

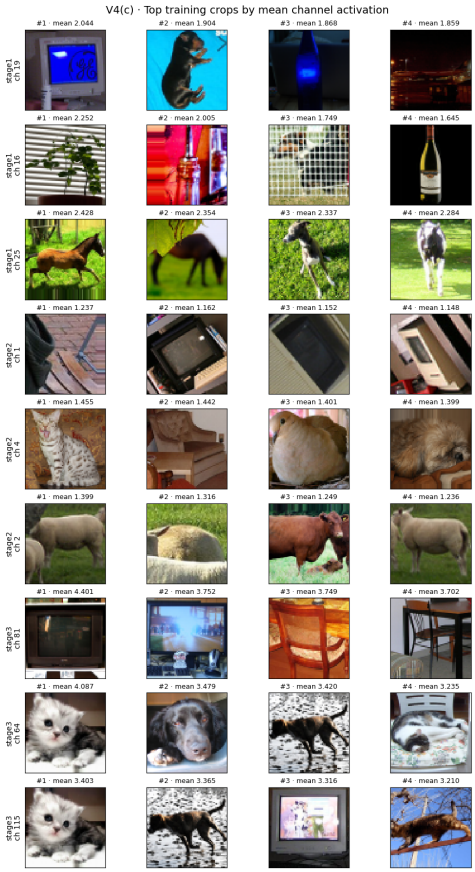
All three models do worse on our photos than on VOC (Tab. 3): A2 drops from 45.7% to 27.7%, A4 from 61.4%



(a) first-layer kernels



(b) three channels per stage on one of our photos



(c) top four training crops per channel

Figure 5. V4: (a) eight first-layer kernels of A4 (after training, and their change from initialization), the Gabor bank, and ResNet-18; (b) three channels per stage on one of our photos; (c) the four training crops without people that activate each channel most.

to 41.7%, and A6-H from 93.5% to 86.0%. A6-H is still clearly the best, and it gets every car and cat right. Its seven mistakes are easy to explain when we look at the photos (Fig. 7). Two photos of the Lime e-bike were called motorbike, which makes sense since it has a thick frame and a basket. Two Smart Water bottles were called chair and two Poland Spring bottles were called diningtable. In those photos the bottle is small and stands on a big white table with a chair behind it, so the table and the chair take up more of the square than the bottle. One photo of the pink office chair was called diningtable because the desk is in the frame. A2 and A4 are much worse on cats (8%, they mostly say chair,

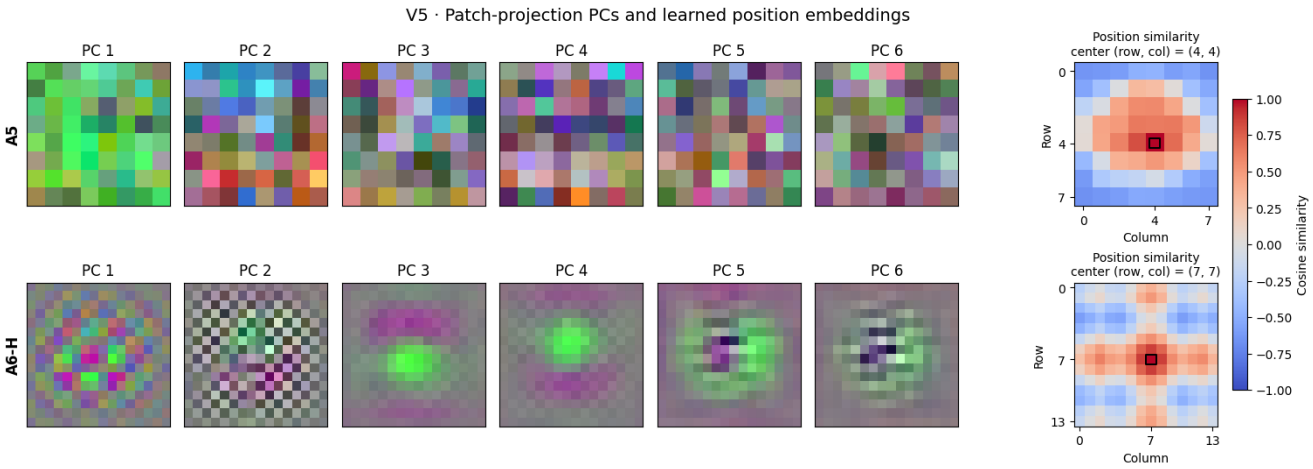


Figure 6. V5: the first six principal components of the patch-projection filters, and the cosine similarity of a central patch's position embedding to all others, for A5 (top) and A6-H (bottom).

	mean	bicycle	bottle	chair	car	cat
A2, B1	27.7	40	20	60	10	8
A2, VOC	45.7	48.0	36.3	53.3	60.9	30.1
A4, B1	41.7	40	20	50	90	8
A4, VOC	61.4	61.0	67.6	56.0	85.7	36.5
A6-H, B1	86.0	80	60	90	100	100
A6-H, VOC	93.5	92.4	96.6	87.4	94.3	96.9

Table 3. B1: accuracy (%) on our photos and on the VOC test crops of the same five classes; "mean" is the mean over these five classes.

tvmonitor, or bottle) and A2 is bad on cars (10%, mostly boat or bird).

The main difference from VOC is framing. A VOC crop is cut from the annotated box, so the object fills most of it. Our photos are whole scenes: after the center crop, the object often covers less than half of the square, and the rest is a table, a street, or a room. Our lighting is also different, with strong autumn sun outside and window light inside. The 64 pixel models (A2, A4) suffer more, since a small object is only a few pixels wide at that size.

Predictions (B2). These are our milestone predictions, copied without changes:

> Our third factor is distance (near and far). Some of our objects, the parked cars and bicycles, are outside, where it is hard to control the lighting, but it is easy to step back and take the same shot from farther away.
>
> We think rotation will cause the largest drop for A6-H, especially at 90° and 180°. Almost all VOC and ImageNet photos are upright, and the only augmentation we use is a horizontal flip. For occlusion, half of the object is still visible, and for distance, the object is still several patches wide at 224 pixels.
>
> Rotation: we think A2 will be the most robust, because the HSV histogram does not change when the image is rotated, and its baseline is already lower, so it has less to lose. A4 and A6-H both depend on the object being upright.
>
> Occlusion: we think A6-H will be the most robust, because attention can use the part that is still visible, and it saw a lot of cluttered and partly hidden objects during pretraining. The plain paper changes the global color histogram for A2 and the pooled features for A4 a lot.
>
> Distance: we think A6-H will be the most robust, because a far object still covers several patches at 224 pixels and pretraining included objects at many sizes. At 64 pixels a far object only covers a few HOG cells or a few positions in A4's feature maps, so the background takes over.

Compared with B3 (Fig. 8):

- Largest A6-H drop: wrong. We guessed rotation, but distance hurt A6-H the most (0.9 to 0.5). Rotation was second (0.6 at 90°). We underestimated how small a far object becomes after the center crop, and how much the table or room around it takes over.
- Rotation, most robust: we cannot really say we were right. A2 (0.2, 0.1, 0.2, 0.0) and A4 (0.1, 0.1, 0.3, 0.1) barely change, but that is because they are wrong on almost every B3 photo already, even at the baseline. A6-H drops the most in points but is still the best at every angle.
- Occlusion, most robust: right. A6-H only goes from 0.9 to 0.8 at both quarter and half. A2 and A4 stay around 0.1 to 0.2.
- Distance, most robust: partly. A6-H is still the best at the far distance (0.5 vs. 0.1 and 0.0), but it has the largest

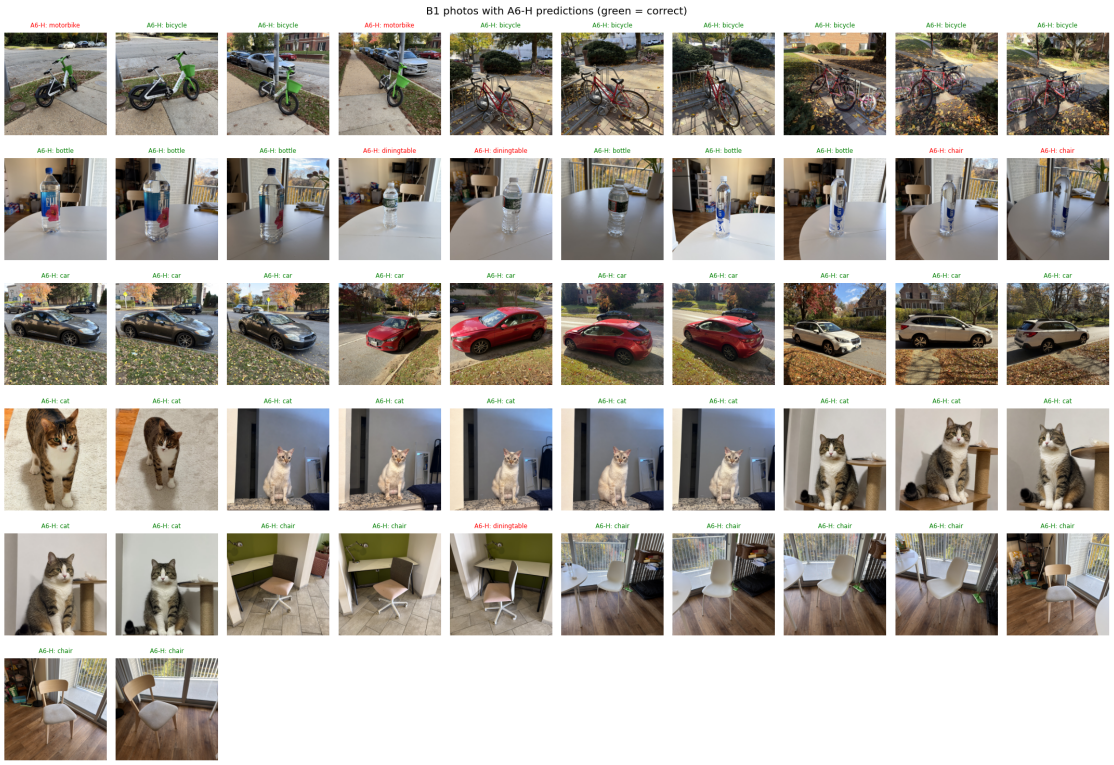


Figure 7. B1: our photos with A6-H's prediction (green right, red wrong).

drop, so our reason ("a far object still covers several patches") was too optimistic.

Controlled series (B3). We photographed two objects per class (10 objects). For each object we kept the phone in one place and took a baseline photo, a photo with about a quarter of the object covered by paper, one with about half covered, and a far photo where the object is much smaller in the frame. The 30°, 90°, and 180° rotations are made in code from the baseline. 90° and 180° are exact, and 30° fills the corners by repeating the border. With two objects per class, each accuracy can only move in steps of 0.1, and with only 10 objects these results describe our objects, not robustness in general. Some photos are not as clean as we planned: the "quarter" occlusion only covers about 10–15% of both chairs and of redbike1, and the far photo of redbike1 has another bike in the foreground. So the quarter results for those objects are a weaker test than planned.

Figure 8 and Tab. 4 show the results. A2 and A4 are near chance on these photos at every condition (0.0 to 0.3), so most of the story is about A6-H. A6-H is perfect on both red bikes and the Siamese cat in every condition. The FIJI bottle is wrong in every condition, always as diningtable. Its B3 photos were taken from farther away than its B1 photos, so even the baseline shows mostly table. For rotation, 90° is the worst (0.6): both cars fail there (bottle and dog), but they are right again at 180°. A car turned 180° is still a wide, horizontal shape, while at 90° it becomes tall and thin. The 30° drop is small (0.8, only the Poland bottle fails), so the border padding does not seem to matter much. For occlusion, only the white chair breaks (sofa at quarter and half). Distance is the biggest problem: at far, the Poland bottle and the white chair become diningtable, the wood chair becomes sofa, and the tabby becomes pottedplant. In all of these the object is small and a larger piece of furniture or the room fills the square.

Minimal pairs (B4). We compared each changed B3 photo with the baseline of the same object and kept pairs where A6-H is right on one photo and wrong on the other, using three different objects and at most one rotation. We found three real flips, so no pair had to be filled in. Figure 9 shows both photos of each pair, A6-H's top three classes, and whether A2 and A4 are right.

Dark car, rotated 90°. What changed: only the rotation, made in code, so this is the cleanest pair. A6-H goes from car 1.00 to bottle 0.33, train 0.23, and motorbike 0.21, so it is not confident about anything. Likely reason: the training data and augmentation. Almost every VOC and ImageNet car is upright and we only used horizontal flips, so

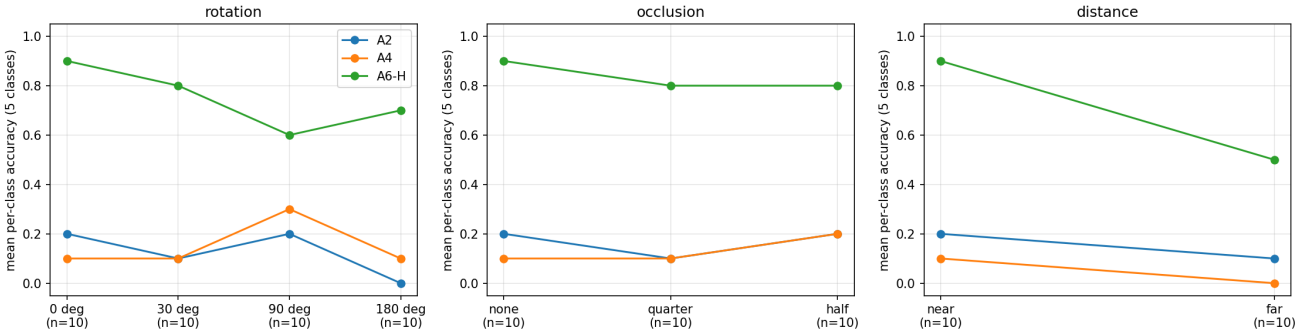


Figure 8. B3: mean per-class accuracy over our five classes for each condition of rotation, occlusion, and distance, for A2, A4, and A6-H; n is the number of photos per condition.

object	base	30°	90°	180°	$\frac{1}{4}$ occ.	$\frac{1}{2}$ occ.	far
bicycle/redbike1	6	6	6	6	6	6	6
bicycle/redbike2	6	6	6	6	6	6	6
bottle/poland	6	–	2,4	–	6	6	–
bottle/fiji	–	–	2,4	–	–	–	–
chair/woodchair	2,6	4,6	4,6	6	6	2,6	2
chair/whitechair	2,6	6	6	4	2	2,4	–
car/darkcar	4,6	6	–	6	4,6	4,6	6
car/whitesuv	6	6	–	6	6	6	6
cat/siamese	6	2,6	6	6	6	6	6
cat/tabby	6	6	6	6	6	6	–

Table 4. B3 per object: which models are right for each object and condition (2 = A2, 4 = A4, 6 = A6-H, – = none), from `b3_per_object.csv`.

the model never saw a car standing on its side. The evidence is that both cars fail at 90° but both are right at 180°, where the outline is still wide. The guesses (bottle, train) are tall or long objects, which suggests the model uses the overall shape and direction of the object. A4 is right on the baseline and wrong after rotation too.

Poland bottle, far. What changed: the distance, but we also moved the phone, so more of the room and the table edge are in the frame. We cannot tell which of these caused the flip. A6-H goes from bottle 0.96 to diningtable 0.98. Likely reason: image formation plus the training data. After the center crop and resize to 224, the bottle is only a few patches wide, while the white table covers about half of the image. A VOC crop of a dining table often has bottles and cups on it, so "a big table with a small bottle" looks like a diningtable crop. The same thing happens in B1 with the Poland and Smart bottles.

Wood chair, far. What changed: the distance, and the far photo also shows part of the white table, the shelf, and the mop. Again more than one thing changed. A6-H goes from chair 1.00 to sofa 0.61, diningtable 0.19, chair 0.18, so chair is still third. Likely reason: the chair is small and the cushioned seat looks like a sofa at low resolution, and the table on the right adds diningtable. Interestingly, A2 (HSV, HOG, Gabor) is right on both photos, maybe because the color and texture of wood and fabric do not depend much on size.

One repair (B5). A6-H dropped the most for distance at far (0.5, compared to 0.9 at near), so we repaired that condition. We set everything before the run: fine-tune A6-H with AdamW, one learning rate of 10^{-5} for all parameters, weight decay 10^{-2}, random flips, and the same 19-way head. Each update uses 8 random VOC training crops (no repeats in a batch) and 8 adaptation photos drawn with replacement, and we stop after 100 updates and keep the final model.

We have to be honest about the photos, because we could not take a new batch in time and used photos we already had. The 10 adaptation photos are far photos of objects that are already in B3: the far photos of both bikes, both bottles, both chairs, and both cars, and two far photos of the Siamese cat. This breaks the rule that adaptation objects should be new. The held-out objects were never used in the repair: the Lime bike, the Smart bottle, the pink chair, and the red car (B1 objects, each with a normal and a far photo taken at the same spot), and the tabby cat (its B3 near and far photos, since we did not have another cat with a far photo). We did not look at the held-out predictions before the run was fixed.

Table 5 shows the results. Because the adaptation photos are the B3 far photos, the jump on B3 far (0.5 to 0.9) mostly shows that the model learned those exact photos, and the B3 and B1 rows are also inflated. So our main evidence is the held-out far row, which goes from 0.4 to 0.8. The Lime bike (motorbike to bicycle) and the Smart bottle (diningtable to bottle) are fixed, the car and chair stay right, and the tabby cat is still pottedplant. This is one photo per class, so one photo moves the row by 0.2, and we only trust the direction: the repair seems to help on new far objects too. The held-out ordinary photos stay at 1.0, so the model did not get worse at normal distances. The cost is small but real: VOC



Figure 9. B4: three minimal pairs. Each row is one object, baseline (left) and one change (right), with A6-H's top three classes and softmax scores and whether A2 and A4 are right.

evaluation set	n	original A6-H	repaired
B3, distance far*	10	50.0	90.0
B3, all images (each once)*	70	72.9	95.7
held-out, ordinary	5	100.0	100.0
held-out, far	5	40.0	80.0
B1*	52	86.0	98.0
VOC test (19 classes)	8381	92.2	91.7

Table 5. B5: mean per-class accuracy (%, over our five classes, except VOC test) before and after one repair run. *These sets contain the adaptation photos or the same objects, so they are not a fair test.

test accuracy goes down from 92.2% to 91.7%, because the model is pulled a bit toward our kind of scenes. With new adaptation objects, as the handout asks, we expect the B3 gain to be smaller than what we report.

model	VOC (19)	B1	total params	trainable	cost
frozen + linear head	88.6	90.0	5,528,083	3,667	1072† + 0.8 s
A6-H, fine-tuned	92.2	86.0	5,528,083	5,528,083	167 s

Table 6. Frozen features vs. fine-tuning (A6-H before the B5 repair), mean per-class accuracy in %: VOC test over 19 classes, B1 over our five classes. Cost is training time: feature extraction plus head fitting for the frozen model, fine-tuning for A6-H (on a Tesla T4). †The frozen run used the Colab CPU by mistake, so its extraction time is not comparable to the T4 time.

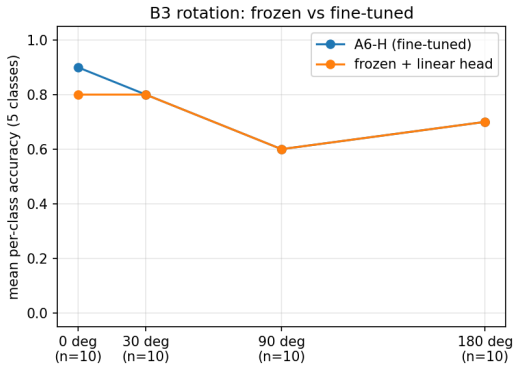


Figure 10. B3 rotation series: mean per-class accuracy over our five classes at each angle, for the frozen-feature model and A6-H (10 photos per angle).

4. Frozen features or fine-tuning? (665 only)

Here we ask what we gain or lose by fine-tuning the whole pretrained ViT instead of only training a linear head on top of it. We loaded the original checkpoint (`vit_tiny_patch16_224.augreg_in21k_ft_in1k`, before any VOC fine-tuning), froze all of it, including the normalization layers and position embeddings, and put it in evaluation mode. The feature is the final normalized class token (192-d), which is the input of the head. We extracted it once for every training crop and once for its horizontal flip, and once (not flipped) for validation, test, and our photos. Then we trained a new 19-way linear head, created right after `torch.manual_seed(1)`, with A6-H's head settings: cross-entropy, AdamW, learning rate 10^{-3}, weight decay 10^{-2}, batch 32, 5 epochs, and the best validation epoch. In each epoch, every training crop uses its normal or flipped feature with probability 0.5. Only the head is trained. This is our milestone prediction, copied without changes:

> 665: we think fine-tuning (A6-H) will beat the frozen features by about 3–8 points on VOC test, because the frozen class token was trained for ImageNet classification and a linear head alone cannot change which features it uses.

On VOC, fine-tuning wins: A6-H gets 92.2% and the frozen model 88.6%, a gap of 3.6 points, so our prediction (3–8 points) was right. The frozen class token was trained for ImageNet's 1000 classes, and a linear head can only reweight it, while fine-tuning can also change the features for VOC crops. But the frozen model is much cheaper to train. It only learns 3,667 parameters (the head), and fitting the head takes 0.8 s, because the features are computed once. A6-H trains all 5.5M parameters with forward and backward passes for 5 epochs (167 s on a T4). Feature extraction is only one forward pass per image, so it should be much faster than fine-tuning on the same GPU.

On our own photos the order flips: the frozen model gets 90.0% on B1 and A6-H 86.0%. This is only 52 photos, so a few photos make the difference, but it fits the idea that fine-tuning on VOC crops makes the model fit VOC framing a bit more, while the frozen ImageNet features stay more general. In the rotation series (Fig. 10) the two models are almost the same (0.8, 0.8, 0.6, 0.7 vs. 0.9, 0.8, 0.6, 0.7). Both fail on the same kinds of photos, the bottles on the table and a car at $90°$, so fine-tuning did not make the model more or less sensitive to rotation. That makes sense, since both models only saw upright images with horizontal flips.

References

[1] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. The PASCAL visual object classes (VOC) challenge. *International Journal of Computer Vision*, 88(2):303–338, 2010.

Contributions

Hanyu Qiu took all of the photos (B1, B3, and B5), ran and analyzed Part B (B1–B5), did the 665 extension (frozen features vs. fine-tuning), and wrote the Part B and 665 sections. Chongxu He did everything else: the data setup, the code and training for all Part A models (A1–A6), the visualizations V1–V5, and the Setup and Part A sections.